



C++ - 모듈 03

계승

요약:

이 문서에는 C++ 모듈의 모듈 03 연습이 포함되어 있습니다.

버전: 7

내용물

나	소개	2
II	일반 규칙	삼
III	연습 00: 아아아아... 열려요!	5
IV	연습 01: 세레나, 내 사랑!	7
V	연습 02: 반복 작업	8
VI	연습 03: 이제 이상해요!	9
VII	제출 및 동료 평가	11

제1장

소개

C++는 Bjarne Stroustrup이 C 프로그래밍 언어 또는 "C with Classes"의 확장으로 만든 범용 프로그래밍 언어입니다(출처: [위키피디아](#)).

이 모듈의 목표는 다음을 소개하는 것입니다. **객체 지향 프로그래밍**. 이것이 C++ 여정의 출발점이 될 것입니다. OOP를 배우려면 많은 언어를 권장합니다. C++는 오랜 친구인 C에서 파생되었기 때문에 선택하기로 결정했습니다. C++는 복잡한 언어이고 단순함을 유지하기 위해 코드는 C++98 표준을 준수합니다.

우리는 현대 C++가 여러 측면에서 많이 다르다는 것을 알고 있습니다. 따라서 능숙한 C++ 개발자가 되고 싶다면 42 Common Core 이후에 더 나아가는 것은 여러분에게 달려 있습니다!

제2장

일반 규칙

컴파일 중

- 다음을 사용하여 코드를 컴파일하세요. `g++` 그리고 깃발 `-Wextra -Werror`
- 플래그를 추가해도 코드는 계속 컴파일되어야 합니다. 표준=C++98

형식 및 명명 규칙

- 연습 디렉토리의 이름은 다음과 같이 지정됩니다. `ex00`, `ex01`, ... , 엑스
- 지침에서 요구하는 대로 파일, 클래스, 함수, 멤버 함수 및 속성의 이름을 지정하십시오.
- 클래스 이름을 적습니다. **어퍼카멜케이스**체제. 클래스 코드가 포함된 파일의 이름은 항상 클래스 이름에 따라 지정됩니다. 예를 들어:
클래스이름.hpp/클래스이름.h, 클래스이름.cpp, 또는 클래스명.hpp. 그런 다음 벽돌 벽을 나타내는 "BrickWall" 클래스 정의가 포함된 헤더 파일이 있는 경우 해당 이름은 다음과 같습니다. `BrickWall.hpp`.
- 달리 지정하지 않는 한, 모든 출력 메시지는 개행 문자로 끝나야 하며 표준 출력에 표시되어야 합니다.
- *안녕 노르미네!* C++ 모듈에는 코딩 스타일이 적용되지 않습니다. 당신이 좋아하는 것을 따라갈 수 있습니다. 하지만 동료 평가자가 이해할 수 없는 코드는 채점할 수 없는 코드라는 점을 명심하세요. 깨끗하고 읽기 쉬운 코드를 작성하기 위해 최선을 다하십시오.

허용/금지

더 이상 C로 코딩하지 않습니다. C++를 배울 시간입니다! 그러므로:

- 표준 라이브러리의 거의 모든 것을 사용할 수 있습니다. 따라서 이미 알고 있는 내용을 고수하는 대신 익숙한 C 함수의 C++ 버전을 최대한 많이 사용하는 것이 현명할 것입니다.
- 그러나 다른 외부 라이브러리는 사용할 수 없습니다. 이는 C++11(및 파생 형식)을 의미하며 후원도서관은 금지되어 있습니다. 다음 기능도 금지됩니다. `*printf()`, `*alloc()` 그리고 무로(`).` 사용하면 성적이 0이 되고 그게 전부입니다.

- 별도로 명시하지 않는 한, 네임스페이스 <ns_name> 사용 그리고 친구 키워드는 금지되어 있습니다. 그렇지 않으면 성적은 -42가 됩니다.
- **모듈 08과 09에서만 STL을 사용할 수 있습니다.** 즉, **아니오 컨테이너**(벡터/목록/맵 등) 및 **아니오 알고리즘**(<를 포함해야 하는 모든 것 알고리즘> 헤더) 그때까지. 그렇지 않으면 성적은 -42가 됩니다.

몇 가지 디자인 요구 사항

- C++에서도 메모리 누수가 발생합니다. 메모리를 할당할 때(새로운 키워드)를 피해야 합니다. **메모리 누수**.
- 모듈 02부터 모듈 09까지 수업은 다음과 같이 설계되어야 합니다. **달리 명시적으로 언급된 경우를 제외하고 정교회 정경 형식**.
- 헤더 파일에 있는 모든 함수 구현(함수 템플릿 제외)은 실습에서 0을 의미합니다.
- 각 헤더를 다른 헤더와 독립적으로 사용할 수 있어야 합니다. 따라서 필요한 모든 종속성을 포함해야 합니다. 그러나 다음을 추가하여 이중 포함 문제를 피해야 합니다. **경비원을 포함하다**. 그렇지 않으면 성적은 0이 됩니다.

나를 읽어주세요

- 필요한 경우(예: 코드 분할) 몇 가지 추가 파일을 추가할 수 있습니다. 이러한 과제는 프로그램에 의해 확인되지 않으므로 필수 파일을 제출하는 한 자유롭게 확인하십시오.
- 때로는 연습 지침이 짧아 보이지만 지침에 명시적으로 기록되지 않은 요구 사항이 예제를 통해 표시될 수 있습니다.
- 시작하기 전에 각 모듈을 완전히 읽으십시오! 정말로, 그렇게 하세요.
- 오딘이, 토르가! 머리를 써라!!!



많은 클래스를 구현해야 합니다. 즐겨 사용하는 텍스트 편집기를 스크립트로 작성할 수 없다면 이는 지루해 보일 수 있습니다.



연습을 완료할 수 있는 어느 정도의 자유가 주어집니다. 그러나 필수 규칙을 따르고 게으르지 마십시오. 당신은 많은 유용한 정보를 놓칠 것입니다! 주저하지 말고 이론적 개념에 대해 읽어보세요.

제3장

연습 00: 아아아아... 열려요!

	운동 : 00
아아아아... 열려요!	
제출 디렉토리: 전-00/	
제출할 파일: Makefile, main.cpp, ClapTrap.{h, hpp}, ClapTrap.cpp	
금지된 기능: 없음	

먼저, 클래스를 구현해야 합니다! 얼마나 독창적입니까!

그것은 불릴 것이다 **시시한** **짓** 대괄호 안에 지정된 값으로 초기화되는 다음과 같은 비공개 속성을 갖게 됩니다.

- 생성자에 매개변수로 전달되는 이름
- 히트 포인트(10)는 ClapTrap의 체력을 나타냅니다.
- 에너지 포인트 (10)
- 공격력 (0)

ClapTrap이 더욱 현실적으로 보이도록 다음 공개 멤버 함수를 추가하세요.

- 무효 공격(const std::string& target);
- void takeDamage(unsigned int amount);
- void beRepaired(unsigned int amount);

ClapTrap이 공격하면 대상이 <공격력>히트 포인트. ClapTrap이 자체적으로 수리되면 <금액>히트 포인트가 돌아옵니다. 공격과 수리에는 각각 에너지 포인트 1이 소모됩니다. 물론 ClapTrap은 체력이나 에너지 포인트가 남아 있지 않으면 아무것도 할 수 없습니다.

이러한 모든 멤버 함수에서는 무슨 일이 일어나는지 설명하는 메시지를 인쇄해야 합니다. 예를 들어, 공격() 함수는 다음과 같이 표시될 수 있습니다(물론 꺾쇠괄호 없이).

ClapTrap <name>이 <target>을 공격하여 <damage>점의 피해를 입힙니다!

생성자와 소멸자도 메시지를 표시해야 동료 평가자가 호출되었음을 쉽게 확인할 수 있습니다.

코드가 예상대로 작동하는지 확인하려면 자체 테스트를 구현하고 설정하세요.

제4장

연습 01: 세레나, 내 사랑!

	운동 : 01
	세레나, 내 사랑!
	제출 디렉토리: 전-01/
	제출할 파일:이전 연습의 파일 + ScavTrap.{h, hpp}, ScavTrap.cpp
	금지된 기능:없음

ClapTrap이 충분할 수 없으므로 이제 파생 로봇을 생성하겠습니다. 이름이 붙을 것이다 **스캐브트랩** Clap-Trap에서 생성자와 소멸자를 상속합니다. 그러나 생성자, 소멸자 및 공격() 다른 메시지를 인쇄합니다. 결국 ClapTraps는 자신의 개성을 알고 있습니다.

테스트에서는 적절한 구성/파괴 체인이 표시되어야 합니다. ScavTrap이 생성되면 프로그램은 ClapTrap을 구축하는 것으로 시작됩니다. 파괴는 역순이다. 왜?

스캐브트랩 ClapTrap의 속성을 사용하고(결과적으로 ClapTrap 업데이트) 다음과 같이 초기화해야 합니다.

- 생성자에 매개변수로 전달되는 이름
- 히트 포인트(100)는 ClapTrap의 체력을 나타냅니다.
- 에너지 포인트 (50)
- 공격력 (20)

ScavTrap에는 고유한 특수 능력도 있습니다.

무효 가드게이트();

이 멤버 함수는 ScavTrap이 이제 게이트 키퍼 모드에 있음을 알리는 메시지를 표시합니다.

프로그램에 더 많은 테스트를 추가하는 것을 잊지 마십시오.

제5장

연습 02: 반복 작업

	운동 : 02
반복적인 작업	
제출 디렉토리: 전-02/	
제출할 파일:이전 연습의 파일 + FragTrap.{h, hpp}, FragTrap.cpp	
금지된 기능:없음	

ClapTraps를 만드는 것이 아마도 당신의 신경을 거슬리기 시작할 것입니다.

이제 **프래그트랩** ClapTrap을 상속받은 클래스입니다. ScavTrap과 매우 유사합니다. 그러나 생성 메시지와 파괴 메시지는 달라야 합니다. 테스트에서는 적절한 구성/파괴 연결이 표시되어야 합니다. FragTrap이 생성되면 프로그램은 ClapTrap을 구축하여 시작됩니다. 파괴는 역순이다.

속성에 대해서도 동일하지만 이번에는 값이 다릅니다.

- 생성자에 매개변수로 전달되는 이름
- 히트 포인트(100)는 ClapTrap의 체력을 나타냅니다.
- 에너지 포인트 (100)
- 공격력 (30)

FragTrap에는 특별한 능력도 있습니다:

```
void highFivesGuys(void);
```

이 멤버 함수는 표준 출력에 긍정적인 하이파이브 요청을 표시합니다.

다시 한번, 프로그램에 더 많은 테스트를 추가하십시오.

제6장

연습 03: 이제 이상해요!

	운동 : 03
이제 이상해!	
제출 디렉토리: 전-03/	
제출할 파일:이전 연습의 파일 + DiamondTrap.{h, hpp}, DiamondTrap.cpp	
금지된 기능:없음	

이 연습에서는 FragTrap의 절반, ScavTrap의 절반인 ClapTrap이라는 괴물을 만듭니다. 이름이 붙을 것이다**다이아몬드트랩**, FragTrap과 ScavTrap 모두에서 상속됩니다. 이건 너무 위험해요!

DiamondTrap 클래스는이름개인 속성. 이 속성에 ClapTrap 기본 클래스의 변수 이름과 정확히 동일한 변수 이름을 지정하십시오(여기서는 로봇 이름에 대해 언급하지 않음).

더 명확하게 설명하기 위해 여기에 두 가지 예가 있습니다.

ClapTrap의 변수가이름,이름을 지어주다이름DiamondTrap 중 하나에. ClapTrap의 변수가 _인 경우이름,이름을 지어주세요 _이름DiamondTrap 중 하나에.

해당 속성과 멤버 함수는 상위 클래스 중 하나에서 선택됩니다.

- 생성자에 매개변수로 전달되는 이름
- 클랩트랩::이름(생성자의 매개변수 + "_박수_이름"접미사)
- 체력(FragTrap)
- 에너지 포인트(ScavTrap)
- 공격 피해(FragTrap)
- 공격() (스캐브트랩)

두 상위 클래스의 특수 기능 외에도 DiamondTrap에는 고유한 특수 기능이 있습니다.

무효 whoAmI();

이 멤버 함수는 이름과 ClapTrap 이름을 모두 표시합니다.

물론 DiamondTrap의 ClapTrap 하위 개체는 한 번만 생성됩니다. 예, 비결이 있습니다.

다시 한번, 프로그램에 더 많은 테스트를 추가하십시오.



-Wshadow 및 -Wno-shadow 컴파일러 플래그를 알고 계십니까?



연습 03을 수행하지 않고도 이 모듈을 통과할 수 있습니다.

제7장

제출 및 동료 평가

과제를 제출하세요. 힘내평소대로 저장소. 방어 중에는 저장소 내부 작업만 평가됩니다. 주저하지 말고 폴더와 파일의 이름이 올바른지 다시 확인하세요.



????????????? XXXXXXXXXXXX = \$3\$cf36316f07f871b4f14926007c37d388